



Full MLOps Revision Notes

1. Why MLOps & Lifecycle (Data → Tracking → Registry → Deployment)

What MLOps is:

MLOps applies DevOps principles (automation, CI/CD, versioning) to the ML workflow. It connects data science experimentation to robust production deployment, ensuring reliability, traceability, and continuous improvement.

Key Lifecycle Stages:

1. Data Management:

Data collection, validation, cleaning, and versioning. Tools like DVC or LakeFS ensure reproducibility of datasets used in experiments.

→ *Goal:* Guarantee that future experiments can reproduce the same training conditions.

2. Experiment Tracking:

Log parameters, metrics, and outputs using **MLflow Tracking**, **Weights & Biases**, etc.

→ *Goal:* Compare model runs and avoid losing track of what worked best.

3. Model Registry:

Central store of versioned models, similar to Git for models. Each version is linked to code, data, and metrics.

→ *Goal:* Manage approval workflows (Staging → Production).

4. Deployment & Monitoring:

Model deployment via REST APIs, containers, or batch jobs. Post-deployment, monitor latency, accuracy, and drift.

→ *Goal:* Continuous learning loop: retrain → redeploy → monitor → improve.

Why MLOps Matters:

Without MLOps, teams face broken reproducibility, manual rework, and model decay. With MLOps, ML becomes scalable, collaborative, and production-grade.

2. Virtualenv & Conda Basics

Virtualenv:

A lightweight Python-only environment isolator.

Commands:

```
python -m venv myenv
source myenv/bin/activate
deactivate
```

Conda:

A more powerful tool that manages both Python and non-Python dependencies (e.g., CUDA, MKL).

Commands:

```
conda create -n myenv python=3.10
conda activate myenv
conda env export > environment.yml
conda env create -f environment.yml
```

When to Use Which:

- **Virtualenv:** For simple Python apps.
 - **Conda:** For ML workloads needing compiled binaries or GPU support.
-

3. Containers vs Virtual Machines

Feature	Containers	Virtual Machines
Isolation	Process-level	Hardware-level
Speed	Starts in seconds	Takes minutes
Kernel	Shares host kernel	Has its own OS kernel
Size	Lightweight (MBs)	Heavy (GBs)
Tool	Docker, Podman	VMware, VirtualBox

Hypervisor:

A layer that enables multiple VMs to share hardware by virtualizing CPU/memory.

Kernel Sharing:

Containers reuse the host OS kernel → faster boot and smaller footprint.

Why Containers for ML:

They package exact runtime (Python, CUDA, libs) ensuring reproducibility, especially across dev → test → prod transitions.

4. Docker Basics: Build, Run, Push, Layers, ENTRYPOINT, FROM

Core Commands:

```
docker build -t myimage:v1 .  
docker run -p 8080:80 myimage:v1  
docker push username/myimage:v1
```

Layers:

Each instruction (RUN, COPY, etc.) adds a cached layer → faster rebuilds.

FROM:

Specifies the base image (e.g., `FROM python:3.11-slim`).

ENTRYPOINT vs CMD:

- `ENTRYPOINT` defines what always executes (e.g., Python app).
 - `CMD` defines default arguments and can be overridden.
-

5. Write a Simple Dockerfile (CentOS + Python + Bash)

```
FROM centos:7  
RUN yum update -y && yum install -y python3 python3-pip bash  
WORKDIR /app  
COPY . /app  
RUN pip3 install -r requirements.txt  
ENTRYPOINT ["python3", "main.py"]
```

Explanation:

- CentOS base ensures consistent environment.
 - Python + pip installed for dependencies.
 - `/app` is working dir.
 - ENTRYPOINT runs the ML app.
-

6. Kubernetes Purpose & Scaling

Purpose:

Kubernetes (K8s) orchestrates containerized workloads—scheduling, scaling, self-healing, and service discovery.

Key Objects:

- **Pod:** Smallest deployable unit (1+ containers).
- **Deployment:** Manages replica sets and rolling updates.
- **Service:** Stable endpoint for accessing pods.
- **HPA (Horizontal Pod Autoscaler):** Scales replicas based on metrics.

Scaling Types:

- **Horizontal:** Add more pods → handles concurrency.
 - **Vertical:** Increase pod resources (CPU/RAM) → handles heavier models.
-

7. Feature Stores (Train–Serve Skew, Batch/Online)

What:

Central database for storing engineered features for ML models.

Types:

- **Offline Store:** For training, large batch data (e.g., BigQuery).
- **Online Store:** Low-latency serving during inference (e.g., Redis).

Train–Serve Skew:

Occurs when training and inference use different feature logic.

→ **Solution:** Feature Store maintains single source of truth.

Examples: Feast, Vertex AI Feature Store, SageMaker Feature Store.

8. AutoML (Ludwig, FLAML)

Goal:

Automate model selection, hyperparameter tuning, and feature preprocessing.

Ludwig:

- YAML-based configuration.
- Supports text, image, tabular data.
- Built on top of TensorFlow/PyTorch.

FLAML (Fast Lightweight AutoML):

- Code-based AutoML by Microsoft.
- Optimizes for cost-efficient, fast tuning.
- Ideal for tabular or regression tasks.

When to Use:

- Rapid baselines or limited ML expertise.

When to Avoid:

- Complex domain-specific problems requiring custom architectures.

9. Cloud ML Tools (Vertex AI, SageMaker, AWS ML Services)

Vertex AI (Google Cloud):

Unified AI platform integrating BigQuery, TPUs, AutoML, and pipelines.

- *Feature Store, Model Registry, Pipelines, and Workbench.*

AWS SageMaker:

All-in-one ML environment with:

- **Studio** (IDE)
- **Model Monitor** (drift detection)
- **Pipelines** (CI/CD)
- **Clarify** (bias detection)
- **JumpStart** (prebuilt models)

AWS Ecosystem Services:

Glue (ETL), S3 (storage), Lambda (serverless inference), CloudWatch (monitoring).

Azure ML:

Strong governance and enterprise integration (Active Directory, Key Vault).

10. MLflow (Tracking URI, start_run, Logging)

Purpose:

Track experiments, log metrics, and manage models in a unified way.

Key Functions:

```
import mlflow
mlflow.set_tracking_uri("http://mlflow.server")
mlflow.set_experiment("demo")
with mlflow.start_run():
```

```
mlflow.log_param("lr", 0.01)
mlflow.log_metric("accuracy", 0.92)
mlflow.log_artifact("model.pkl")
```

Components:

1. **Tracking:** Logs parameters, metrics, artifacts.
 2. **Projects:** Reproducible packaging.
 3. **Models:** Multi-flavor deployment (sklearn, torch).
 4. **Registry:** Manage model lifecycle versions.
-

11. Drift Detection (Data vs Concept)

Data Drift:

Feature distribution changes (e.g., age values now 20–40 instead of 30–60).
→ Measured via PSI (Population Stability Index).

Concept Drift:

The relationship between X and Y changes (e.g., fraud patterns evolve).
→ Detected via accuracy drops or hypothesis tests on prediction errors.

Mitigation:

Monitor metrics, set thresholds, retrain automatically via pipelines.

12. CI/CD Basics, FastAPI Serving, Containers for Microservices

CI/CD (Continuous Integration / Continuous Deployment):

- **CI:** Automated build, linting, tests on every push.
- **CD:** Automatic deploy of passing models/images to staging or production.

FastAPI for Serving:

Lightweight Python framework for REST APIs with built-in validation.

```
from fastapi import FastAPI
app = FastAPI()

@app.post("/predict")
def predict(data: InputSchema):
    result = model.predict(data)
    return {"prediction": result}
```

Containers for Microservices:

Each model packaged as a self-contained Docker image → scalable, testable, and easily rolled back.

13. Conda Export/Recreate Commands

To share or rebuild the same environment:

```
conda env export --no-builds > environment.yml
conda env create -f environment.yml
conda list
conda remove -n myenv --all
```

--no-builds makes the YAML portable across OS and hardware.

Core Idea Summary:

MLOps brings structure to the chaos of ML:

- Reproducible environments (Conda/Docker)
- Scalable deployment (K8s, CI/CD)
- Traceable experiments (MLflow)
- Consistent data (Feature Stores)
- Continuous improvement (Drift detection + retraining).